# CHAPTER 3

# Transport Layer

**3.1 Introduction and Transport-Layer Services**

3.1.1 Relationship between Transport and Network Layers

3.1.2 Overview of the Transport Layer in the Internet

**3.2 Multiplexing and Demultiplexing**

**3.3 Connectionless Transport: UDP**

3.3.1 UDP Segment Structure

**3.4 Principles of Reliable Data Transfer**

3.4.2 Pipelined Reliable Data Transfer Protocols

3.4.3 Go-Back-N (GBN)

3.4.4 Selective Repeat (SR)

**3.5 Connection-Oriented Transport: TCP**

3.5.1 The TCP Connection

3.5.2 TCP Segment Structure

3.5.4 Reliable Data Transfer

3.5.5 Flow Control

3.5.6 TCP Connection Management

**3.7 TCP Congestion Control**

© 2019 – Mohammad MamunElahi

# 3.1 Introduction and Transport-Layer Services

- **Process-to-process communication between applications:** Segmentation, reliability

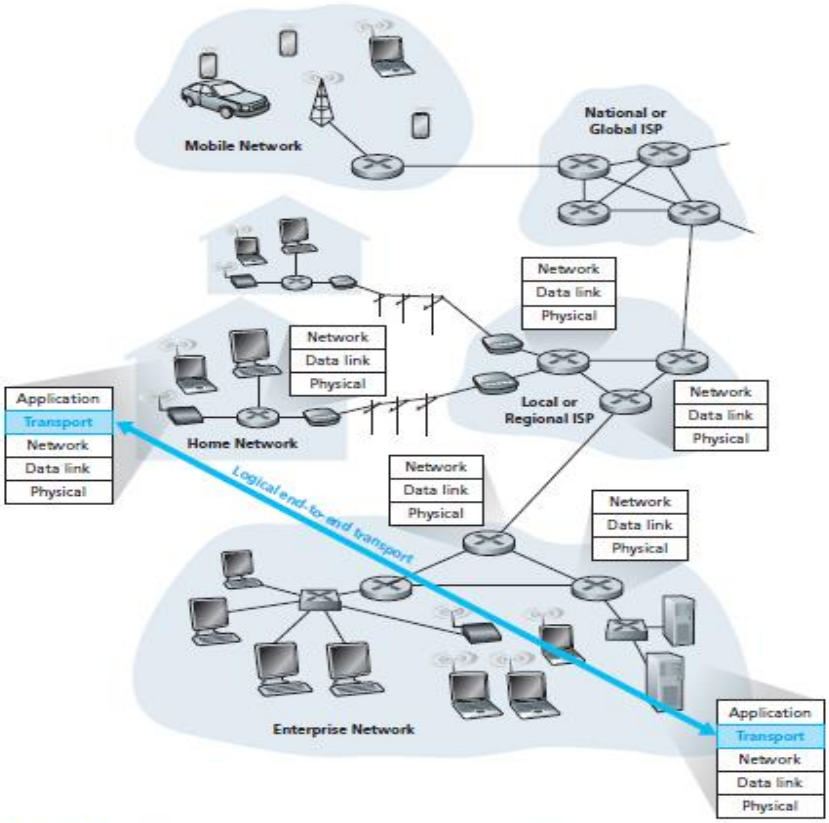
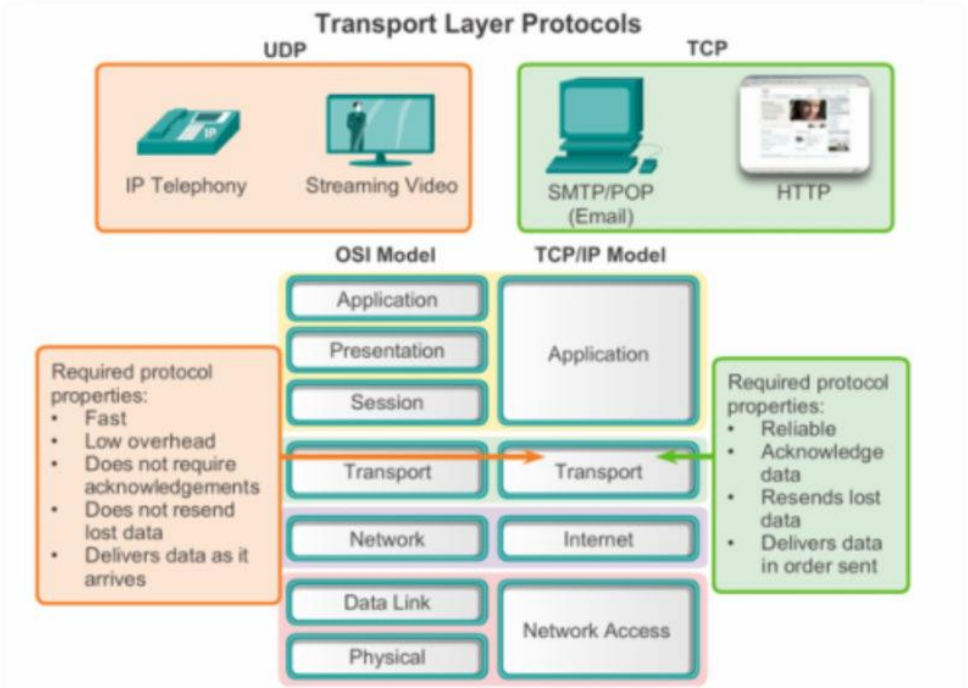



Figure 3.1 ♦ The transport layer provides logical rather than physical communication between application processes

- **Transport layer protocols: TCP** (reliable) and **UDP** (unreliable)



# 3.2 Multiplexing and Demultiplexing (using **sockets** and **port numbers**)

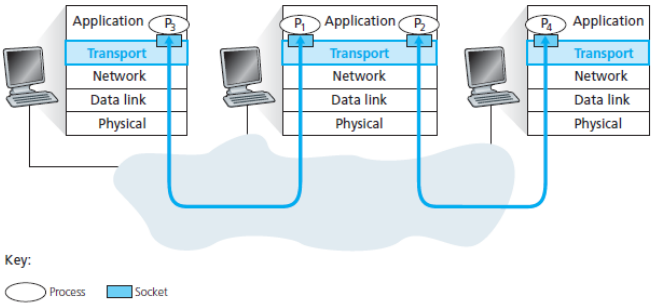



Figure 3.2 ♦ Transport-layer multiplexing and demultiplexing

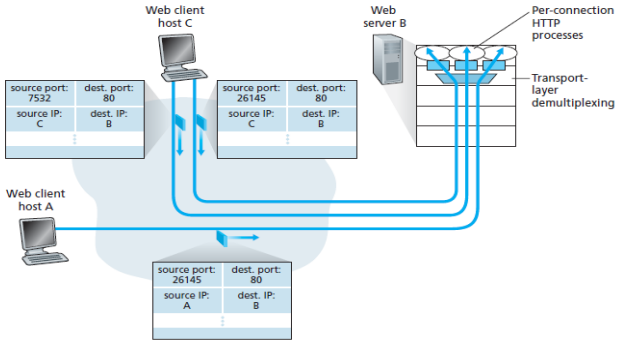
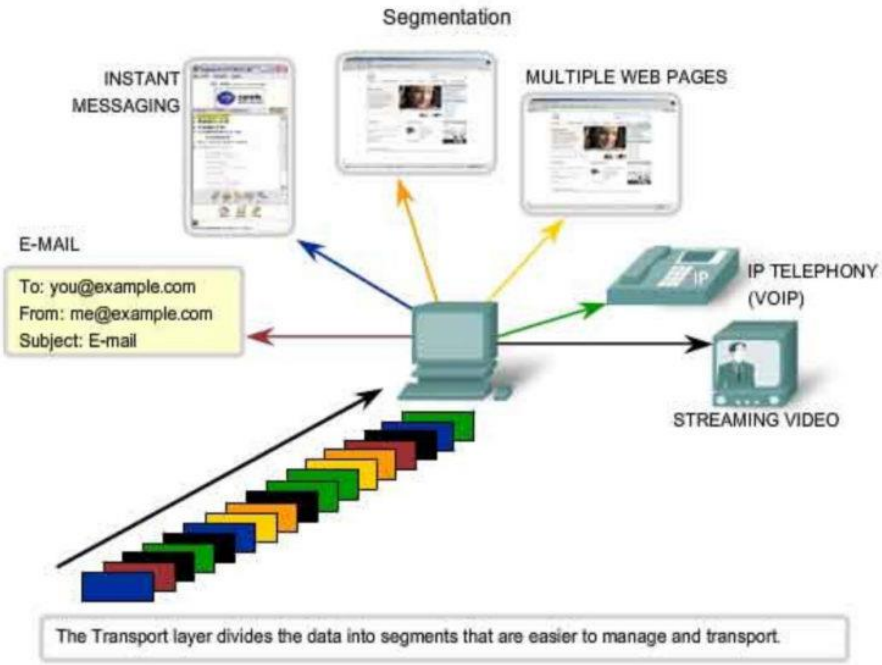
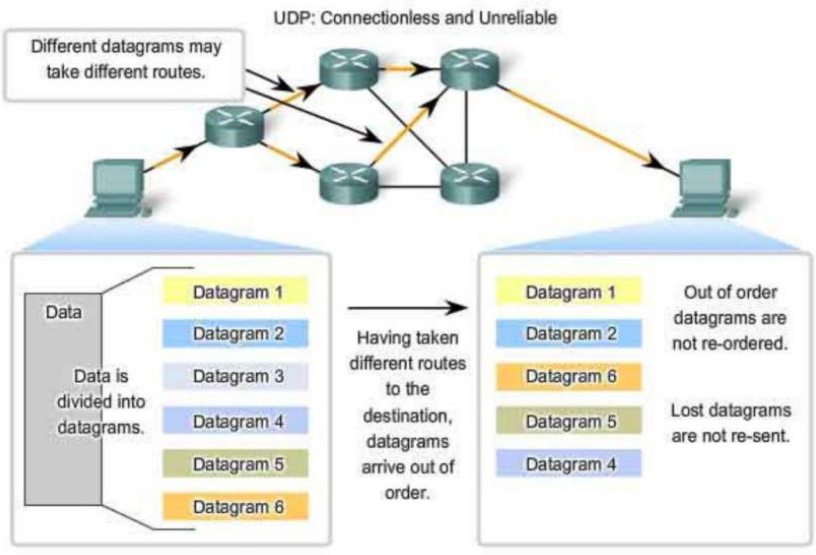



Figure 3.5 ♦ Two clients, using the same destination port number (80) to communicate with the same Web server application




# 3.3 Connectionless Transport: UDP



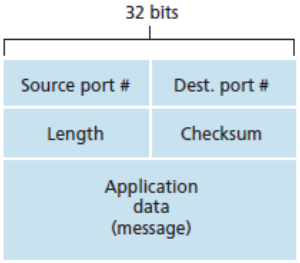




Figure 3.7 ♦ UDP segment structure

# Knowledge Test # 1

**Q.1** What is **basic difference** between **Transport** and **Network** layer? Illustrate with the **"household analogy" example** in the text book.

**Q.2** What are the **two protocols** supported by the transport layer?

**Q.3** Why **multiplexing and demultiplexing** is used? How **socket** and **port numbers** are used to implement multiplexing and demultiplexing?

**Q.4** What are **the reasons** for UDP been faster than TCP? What are the fields in the **UDP segment**?

© 2019 – Mohammad MamunElahi

# 3.4 Principles of Reliable Data Transfer

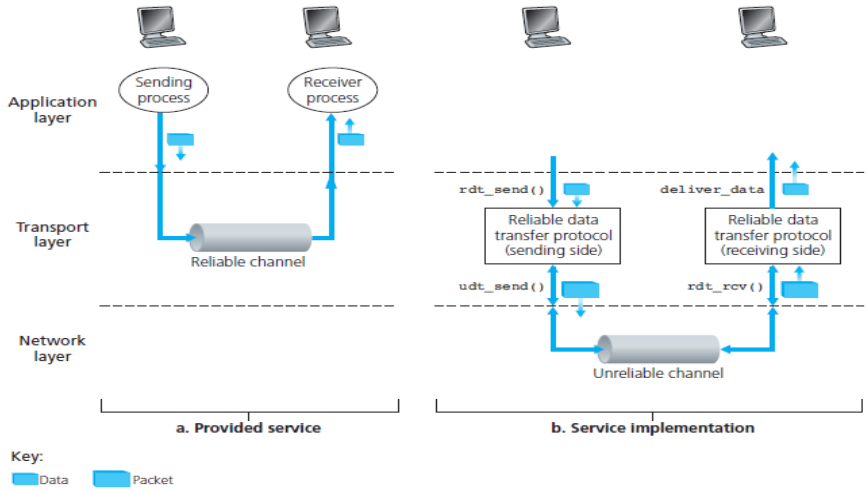



Figure 3.8 ♦ Reliable data transfer: Service model and service implementation

- **Pipelined/non-pipelined reliability protocols**

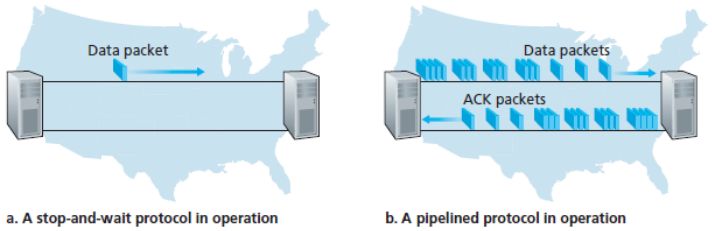
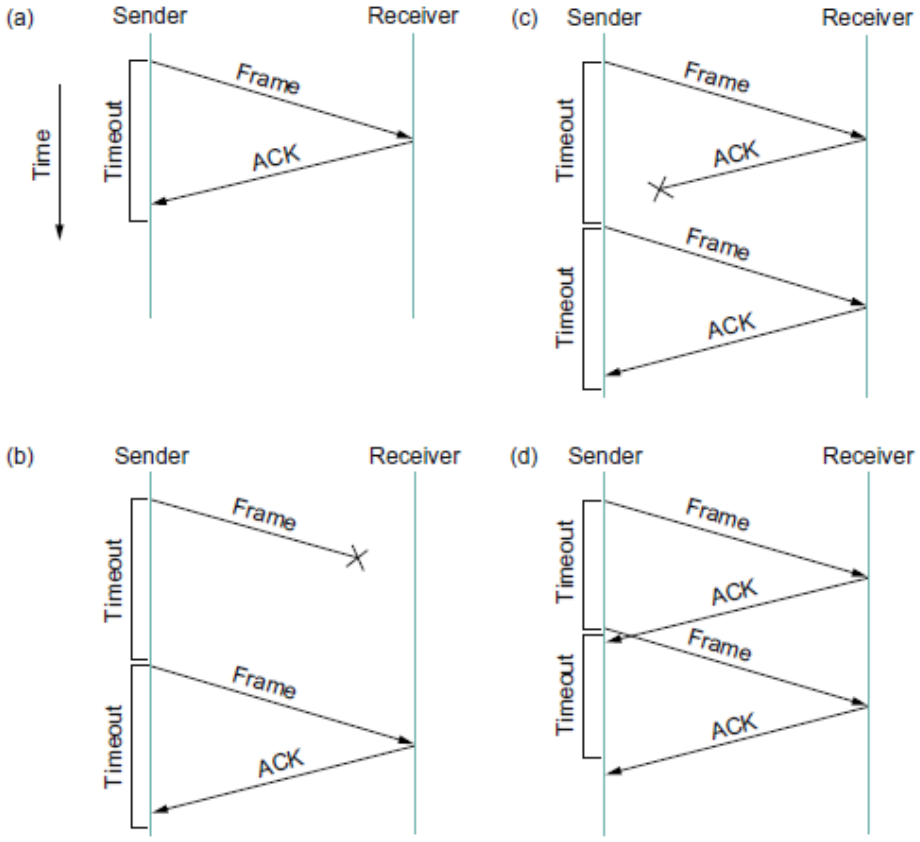



Figure 3.17 ♦ Stop-and-wait versus pipelined protocol




Timeline showing four different scenarios for the stop-and-wait algorithm. (a) The ACK is received before the timer expires; (b) the original frame is lost; (c) the ACK is lost; (d) the timeout fires too soon.

## ❖ Pipelined protocol: Go-back-N

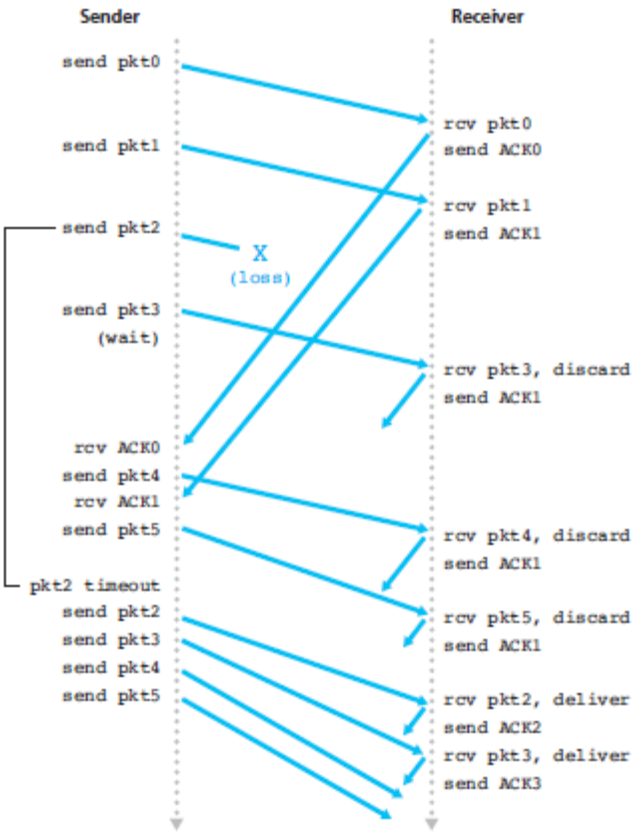



Figure 3.22 ♦ Go-Back-N in operation

## ❖ Pipelined protocol: Selective-repeat

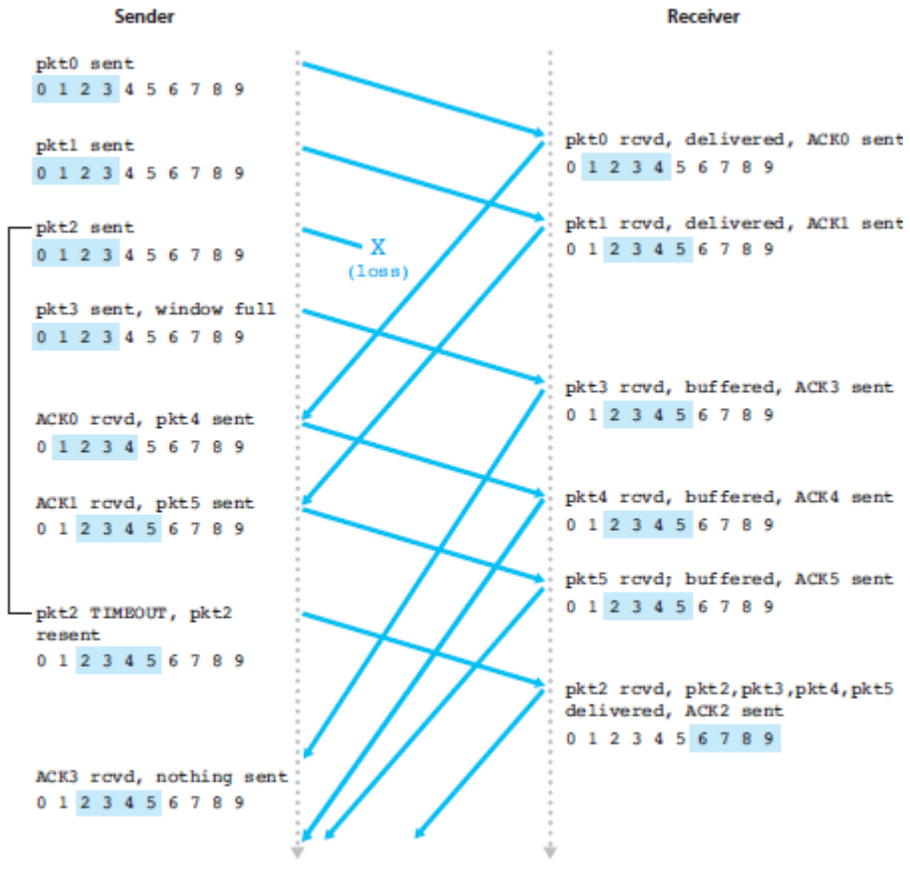
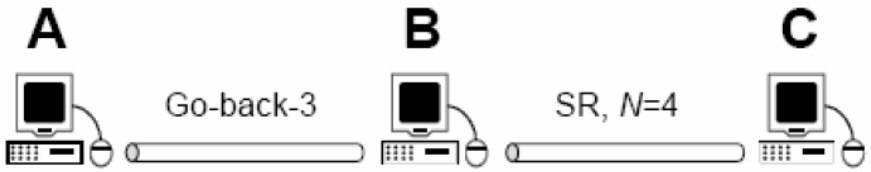



Figure 3.26 ♦ SR operation

❑ **Exercise 1:** Suppose **3 hosts** are connected as shown in the figure. **Host A** sends packets to **host C** through **host B**. However, they use different protocols for reliable communication (Go-Back-N vs. Selective Repeat). Notice that B is a regular host running both as a receiver (to receive packets from A) and sender (to forward A's packets to C).




Draw **side-by-side the timing diagram** for A→B and B→C transmission up to the time when the **first eight packets** from A show up on C. Assume that the **2nd and 4th packets** arrive in error to **host B** on their first transmission, and the **3rd packet** arrives in error to **host C** on its first transmission.

# Knowledge Test # 2

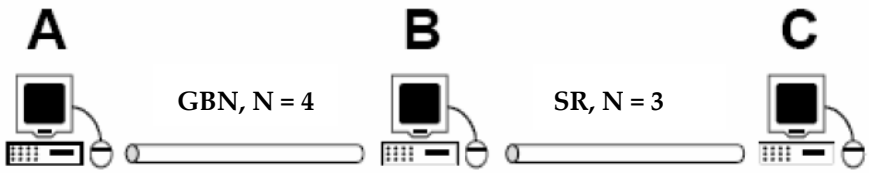
**Q.1** What is meant by the **reliable channel (Transport layer)** and **unreliable channel (Network layer)** in **Figure 3.8** in the textbook?

**Q.2** Give examples of **non-pipelined** and **pipelined** reliable protocols? What is the basic difference among them?

**Q.3** Write down **3 distinct differences** between **Go-back-N** and **Selective-repeat**?

| Go-back-N | Selective Repeat |
|---|---|
| | |
| | |
| | |

**Q.4** Suppose **3 hosts** are connected as shown in the figure. **Host A** sends packets to **host C** through **host B**. However, they use different protocols for reliable communication (Go-Back-N vs. Selective Repeat). Notice that B is a regular host running both as a receiver (to receive packets from A) and sender (to forward A's packets to C).




Draw **side-by-side the timing diagram** for A→B and B→C transmission up to the time when the **first eight packets** from A show up on C. Assume that the **3rd and 5th packets** arrive in error to **host B** on their first transmission, and the **4th packet** arrives in error to **host C** on its first transmission.

# 3.5 Connection-Oriented Transport: TCP

## ❖ Segmentation: Process-to-process communication

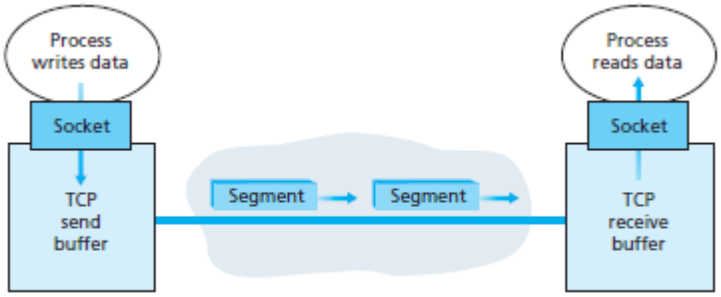



Figure 3.28 ♦ TCP send and receive buffers

## ❖ TCP Segment Structure

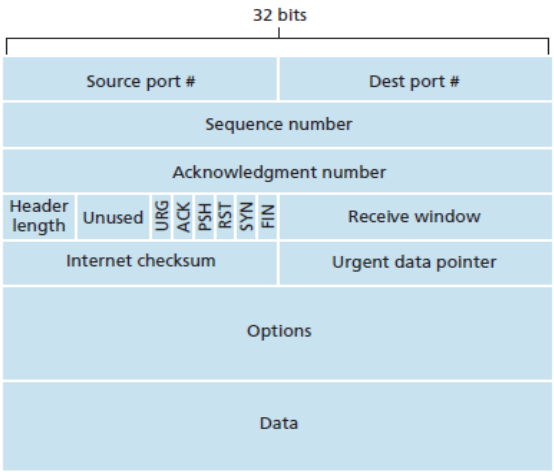



Figure 3.29 ♦ TCP segment structure

## ❖ Some interesting scenarios of data transmission in TCP

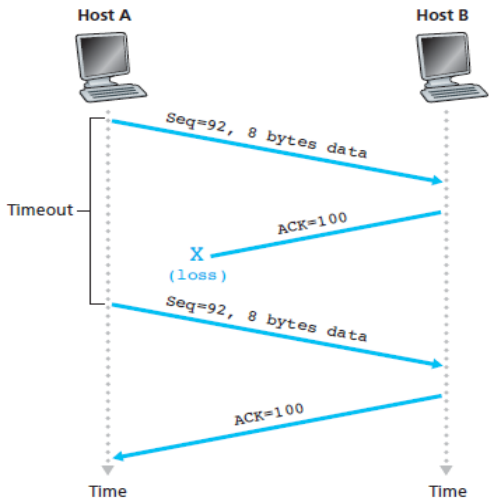



Figure 3.34 ♦ Retransmission due to a lost acknowledgment

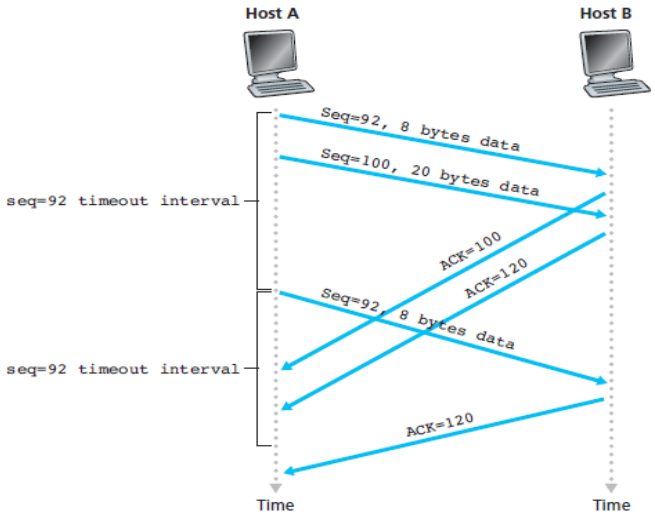



Figure 3.35 ♦ Segment 100 not retransmitted

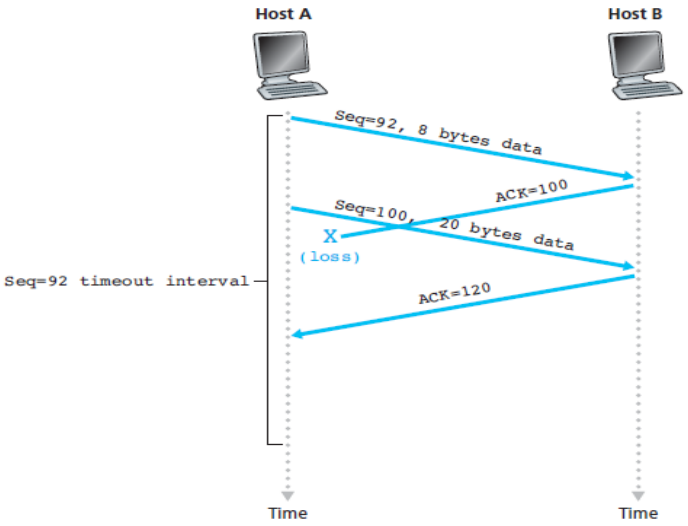



Figure 3.36 ♦ A cumulative acknowledgment avoids retransmission of the first segment

❑ **Exercise 2:** The sender buffer **SendBase** is 328. The **last TCP sequence number** sent was 950 in a segment with 500 bytes of data. A segment is received with **TCP acknowledgement number** 350 and **window** 3000. **MSS** is 512. Until another ACK is received:

a. What is the new value of **SendBase**?

b. What is the **last byte (number)** that can be sent with certainty that the receiver's buffer will not overflow?

c. What is the **next byte (number)** that will be sent?

❑ **Exercise 3:** Suppose **A** sends **two TCP segments** back-to-back to **B**. The first segment has **sequence number 300**, with **150 bytes** of data; the second has **sequence number 550,** with **100 bytes** of data.

a) **Draw the scenario** in a diagram.

b) Suppose that **the first segment is lost**, but the second segment arrives at B. What will be the **acknowledgment number** that B sends to A?

c) Suppose the **two segments** sent by A arrive in order at B. The **first acknowledgement is lost** and the second acknowledgement arrives after the first timeout interval, what will be the **sequence number** of the segment that A retransmits to B? What will be the **acknowledgement number** from B after receiving the retransmitted segment?

❖ **Fast retransmit in TCP**

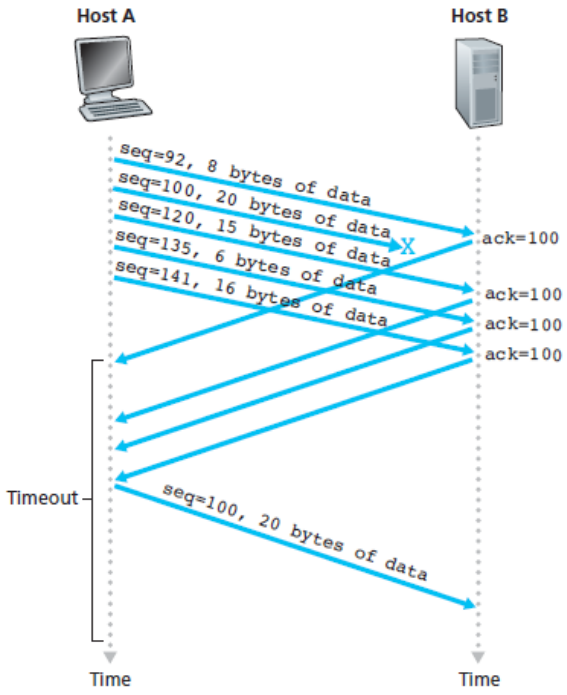



**Figure 3.37** ♦ Fast retransmit: retransmitting the missing segment before the segment's timer expires

## ❖ TCP Flow Control

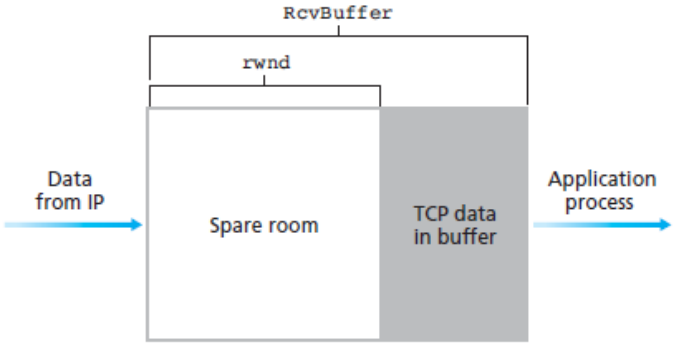



**Figure 3.38** ♦ The receive window (`rwnd`) and the receive buffer (`RcvBuffer`)

## ❖ TCP Connection Management: <u>Connection establish</u>, <u>Connection close</u>

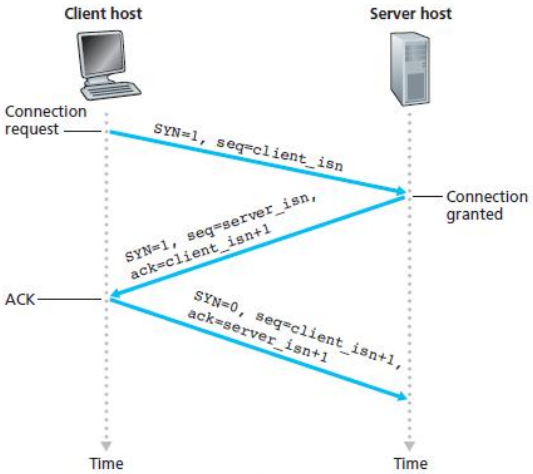



**Figure 3.39** ♦ TCP three-way handshake: segment exchange

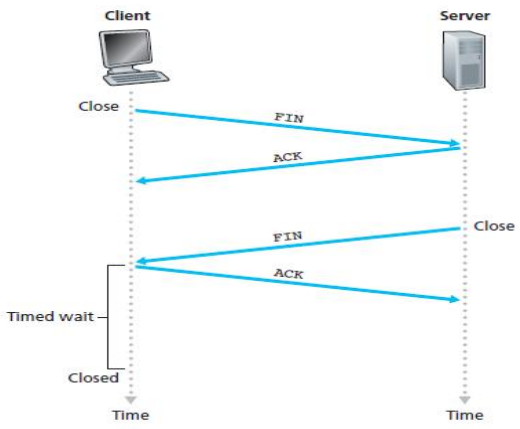
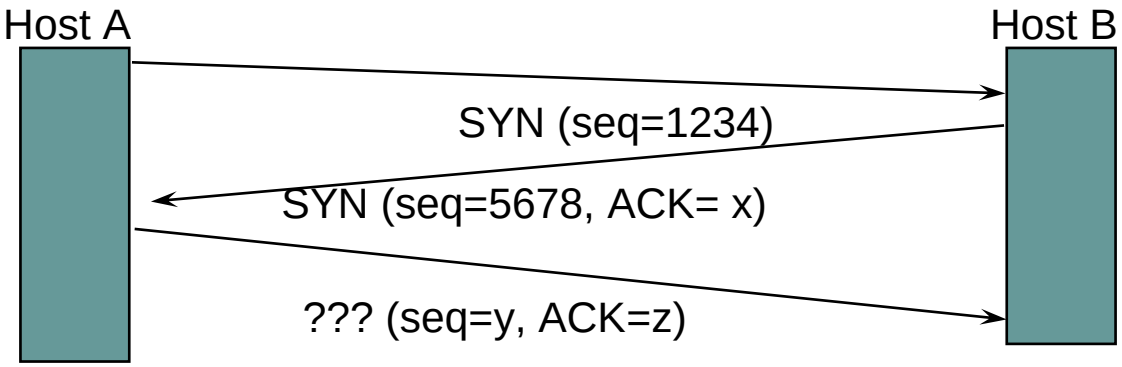



**Figure 3.40** ♦ Closing a TCP connection

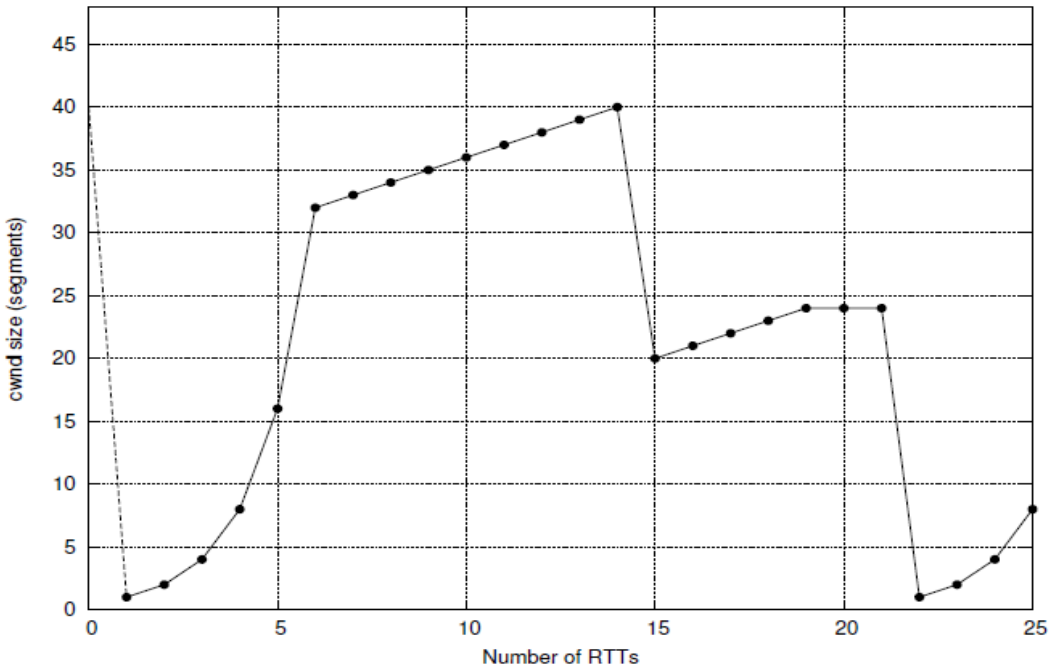
❑ **Exercise 4:** **Two hosts** want to communicate using **TCP** and issue the packets in the following diagram to establish the connection. Give the **packet sequence numbers** that should be inserted in the diagram at the positions indicated by the letters **'x', 'y'** and **'z'**.




## 3.7 TCP Congestion Control

- **3 Primitive Approaches:** **AIMD**, **Slow Start**, **Congestion avoidance**

❑ **Exercise 5:**




Assume **TCP Reno (meaning Fast Retransmit and Fast Recovery)** is the protocol experiencing the behavior shown above. Assume that the TCP flow has been operating for some time, meaning that the number of RTTs shown is with respect to when you started observing the flow's behavior.

a) Identify the **time periods** when **TCP slow start** is operating.
b) Identify the **time periods** when **TCP congestion avoidance** is operating (AIMD).
c) After the **14th RTT**, is the segment loss detected by a **triple duplicate ACK** or by **a timeout**?
d) What is the **initial value of ssthreshold**, before the **first congestion avoidance interval**?
e) What is the value of **ssthreshold** at the **15th RTT**?
f) What is the value of **ssthreshold** at the **22nd RTT**?
g) Assuming a packet loss is detected after the **25th RTT** by the receipt of a **triple duplicate ACK**, what will be the values of the **congesion-window size** and of **ssthreshold**?

# Knowledge Test # 3

**Q.1 True** or **False**?

a) Host A is sending a large file to host B over a TCP connection. Assume host B has no data to send to host A. Host B will not send acknowledgments to host A because host B cannot **piggyback** the acknowledgments on data. [ ***Piggybacking*** *is an optimization that is used when both sides have to send data to each other so that the receiver, instead of sending two packets i.e., an ACK and a data packet, it just sends one. ]*
b) The size of the TCP **RcvWindow** never changes throughout the duration of the connection.
c) Suppose host A is sending a large file to host B over a TCP connection. The number of unacknowledged bytes that A sends cannot exceed the size of the advertised receiver buffer.

**Q.2** Consider the **TCP segment structure** in **Figure 3.29** in the textbook. Why the following fields are used: **Source and destination port numbers**, **sequence and acknowledge numbers**, and **receive window**.

**Q.3** Hosts A and B are communicating over a **TCP connection**. Host B has already received from A all bytes up through byte **248**. Suppose Host A then sends **two segments** to Host B back-to-back. The first and second segments contain **40** and **60 bytes** of data respectively. In the first segment, the sequence number is **249**, the **source port number** is **503**, and the **destination port number** is **80**. Host B sends an **acknowledgement** whenever it receives a segment from Host A.

a) In the second segment sent from Host A to Host B, what are the sequence number, source port number, and destination port number?
b) Suppose the **first segment arrives before the second segment**. Host B sends an acknowledgement in response to the first segment. what is the **acknowledgment number**, the source port number, and the destination port number?
c) Suppose the second segment arrives before the first segment. Host B sends an acknowledgement in response to the first segment. What is the **acknowledgement number**?
d) Suppose the two segments sent by A **arrive in order** at B. The first acknowledgement is lost and the second acknowledgement arrives **before the first timeout interval**. Draw a **timing diagram**, showing these segments and all other segments and acknowledgements sent.

**Q.4** What is **Fast retransmit**? Illustrate with **an example**.

**Q.5** How **flow control** in TCP works? Illustrate with a **diagram**.

**Q.6** How **connection** is **established** and is **closed** in TCP?

**Q.7** What is the difference between **flow control** and **congestion control** in TCP?

**Q.8** What are the 3 (three) primitive **congestion control approaches** in TCP? Illustrate with proper **diagrams**.

© 2019 – Mohammad MamunElahi 